

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №2

Выполнил:

Грищук Елизавета Дмитриевна
БР 2.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Спроектировать REST API для приложения бронирования столиков в ресторанах. Описать функциональные и нефункциональные требования, спроектировать все эндпоинты в формате OpenAPI-схемы с учётом реализованной диаграммы БД, описать тела запросов и ответов, продумать возможные ошибки.

Ход работы

1. Функциональные требования

- Регистрация и авторизация пользователей посредством JWT-токенов
- Просмотр и редактирование профиля пользователя
- Просмотр списка категорий кухни
- Просмотр списка ресторанов с фильтрацией по категории кухни, адресу и ценовому диапазону
- Просмотр детальной информации о ресторане (название, адрес, описание, фото, категория)
- Просмотр списка столиков ресторана с указанием вместимости
- Создание бронирования столика с указанием времени и количества гостей
- Просмотр истории бронирований пользователя
- Получение информации о конкретном бронировании
- Отмена бронирования
- Просмотр отзывов о ресторане
- Создание отзыва о ресторане с оценкой и комментарием

2. Нефункциональные требования

- API реализуется в формате REST
- Аутентификация осуществляется посредством JWT-токенов, передаваемых в заголовке Authorization
- Все запросы и ответы передаются в формате JSON
- API корректно обрабатывает ошибки и возвращает соответствующие HTTP-статусы
- Время ответа API не должно превышать 500 мс при стандартной нагрузке

3. Формат реализации API

Выбран формат REST API. Данный формат является стандартным для веб-приложений подобного типа, обеспечивает простоту разработки и поддержки, а также хорошо совместим с большинством клиентских приложений.

4. Описание эндпоинтов

Спроектированы следующие группы эндпоинтов:

Auth — регистрация и авторизация:

- POST /auth/register — регистрация нового пользователя. Принимает name, email, password. Возвращает объект пользователя. Возможные ошибки: 400 (некорректные данные), 409 (email уже занят), 422 (невалидный формат email или слабый пароль).
- POST /auth/login — авторизация пользователя. Принимает email, password. Возвращает JWT-токен. Возможные ошибки: 401 (неверный пароль), 404 (пользователь не найден).

Users — управление профилем:

- GET /users/me — получение профиля текущего пользователя. Требуется авторизация. Возможные ошибки: 401 (не авторизован), 403 (токен истёк).
- PATCH /users/me — обновление профиля. Принимает name, email. Возможные ошибки: 401, 403, 409 (email занят), 422 (невалидный email).

Categories — категории кухни:

- GET /categories — получение списка всех категорий кухни.

Restaurants — рестораны:

- GET /restaurants — получение списка ресторанов с фильтрацией по category_id, address, price_range. Возможные ошибки: 400 (невалидные параметры).
- GET /restaurants/{id} — получение детальной информации о ресторане. Возможные ошибки: 404 (не найден).
- GET /restaurants/{id}/tables — получение списка столиков ресторана. Возможные ошибки: 404.

Reviews — отзывы:

- GET /restaurants/{id}/reviews — получение отзывов о ресторане. Возможные ошибки: 404.
- POST /restaurants/{id}/reviews — создание отзыва. Принимает rating (1–5), comment. Требуется авторизация. Возможные ошибки: 400, 401, 403, 404, 409 (отзыв уже оставлен).

Bookings — бронирования:

- GET /bookings — история бронирований пользователя. Требуется авторизация. Возможные ошибки: 401, 403.
- POST /bookings — создание бронирования. Принимает table_id, booking_time, guests_count. Требуется авторизация. Возможные ошибки: 400, 401, 403, 404 (стол не найден), 409 (стол уже занят), 422 (guests_count превышает вместимость стола).
- GET /bookings/{id} — информация о конкретном бронировании. Требуется авторизация. Возможные ошибки: 401, 403 (чужое бронирование), 404.
- PATCH /bookings/{id}/cancel — отмена бронирования. Требуется авторизация. Возможные ошибки: 401, 403 (чужое бронирование), 404, 409 (уже отменено).

5. Схемы данных

Описаны следующие схемы: User, Category, Restaurant (включает photo_url), Table, Booking (статусы: pending, confirmed, cancelled), Review, Error.

Полная OpenAPI-схема реализована в формате YAML и доступна в Swagger UI.

Restaurant Booking API 1.0.0 OAS 3.0

API для приложения бронирования столиков в ресторанах

Servers

http://localhost:3000/api

Authorize

Auth

POST

/auth/register

Регистрация пользователя

POST

/auth/login

Авторизация пользователя

Users

GET

/users/me

Получить профиль текущего пользователя

PATCH

/users/me

Обновить профиль текущего пользователя

Categories

GET

/categories

Получить список категорий кухни

Restaurants

POST

/restaurants/{id}/reviews

Оставить отзыв о ресторане

Parameters

Try it out

Name

Description

id * required
integer
(path)

id

Request body required

application/json

Example Value | Schema

```
{
  "rating": 5,
  "comment": "string"
}
```

Responses

Code

Description

Links

201

Отзыв создан

No links

Media type

application/json

Controls Accept header.

Example Value | Schema

Вывод

В ходе выполнения работы были описаны функциональные и нефункциональные требования к REST API приложения бронирования столиков в ресторанах. Спроектированы 13 эндпоинтов, разбитых на 5 групп. Для каждого эндпоинта описаны тела запросов и ответов, а также возможные ошибки с соответствующими HTTP-статусами. Схема реализована в формате OpenAPI 3.0 и проверена в Swagger UI.